

Slip 5

Q.1) Read the current directory and display the name of the files, no of files in current directory

```
#include<stdio.h>
#include<stdlib.h>
#include<dirent.h>
#include<sys/stat.h>
#include<time.h>

int main()
{
    int c=0;
    struct dirent *entery;
    DIR *dp;
    dp=opendir(".");
    if(dp==NULL)
    {
        printf("not open");
    }
    else
    {
        while((entery=readdir(dp))!=NULL)
        {

            printf("\n name of file is : %s",entery->d_name);
            c++;

        }
        closedir(dp);
        printf("total file count is %d ",c);
    }
}
```

Q.2) Write a C program to create an unnamed pipe. The child process will write following three

messages to pipe and parent process display it.

Message1 = "Hello World"

Message2 = "Hello SPPU"

Message3 = "Linux is Funny"

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
int main() {
```

```

int fd[2]; // fd[0] = read end, fd[1] = write end
pid_t pid;
char buffer[100];
// Create unnamed pipe
if (pipe(fd) == -1) {
    perror("pipe");
    exit(1);
}
pid = fork();
if (pid < 0) {
    perror("fork failed");
    exit(1);
}
if (pid == 0) {
    // -----
    // Child Process: write to pipe
    // -----
    close(fd[0]); // Close read end
    char *messages[] = {
        "Hello World",
        "Hello SPPU",
        "Linux is Funny"
    };
    for (int i = 0; i < 3; i++) {
        write(fd[1], messages[i], strlen(messages[i]) + 1); // +1 for null terminator
    }
    close(fd[1]); // Close write end
    exit(0);
} else {
    // -----
    // Parent Process: read from pipe
    // -----
    close(fd[1]); // Close write end
    printf("Parent reading messages from pipe:\n");
    for (int i = 0; i < 3; i++) {
        read(fd[0], buffer, sizeof(buffer));
        printf("%s\n", buffer);
    }
    close(fd[0]); // Close read end
}
return 0;
}

```